

Week 2 Synopsis

Refer template for necessary details – just showing some technical field details for sample

SkillScan: AI-Based Resume Shortlisting System

Abstract

This project aims to automate the resume shortlisting process using advanced Natural Language Processing (NLP) techniques. By analyzing job descriptions and resumes, the system will assign match scores to candidates, significantly reducing the manual efforts of HR teams. The solution includes resume parsing, keyword extraction, and similarity scoring to recommend the most relevant candidates. Developed using Python, Flask, and NLP libraries such as spaCy or BERT, the system will provide an intuitive web interface and backend logic to streamline recruitment.

1. Introduction

1.1 Background and Context

With the growing number of job applicants, manual resume shortlisting has become time-consuming and inefficient. Leveraging AI and NLP can enhance this process by automating resume-job matching.

1.2 Problem Statement

HR teams struggle to shortlist relevant resumes from hundreds of applicants per role. Manual screening is inefficient, inconsistent, and prone to errors.

1.3 Significance of the Project

This project brings efficiency, consistency, and objectivity to the recruitment process. It reduces human effort, saves time, and improves candidate-role alignment.

1.4 Objectives

- Automate the resume shortlisting process using NLP.
- Extract relevant skills and keywords from resumes and JDs.
- Generate match scores for resumes based on JD content.
- Provide a web interface for HR and job seekers.
- Enable filtering, viewing, and exporting of shortlisted candidates.

1.5 Scope and Limitations

Scope: Resume parsing, match score generation, candidate filtering.

Limitations: Focused on English resumes; does not consider image-based CVs or references.

2. Methodology

Include diagrams as applicable based on project

2.1 Technical Approach

Client-server architecture using Flask backend and HTML/CSS frontend. AI logic uses rule-based keyword matching and transformer-based semantic similarity.

2.2 System Architecture

- Frontend: Resume & JD Upload, Results Display
- Backend: Parsing engine, NLP processing, scoring algorithm
- Data Structures: Hash tables, lists, trees (for keyword indexing)

2.3 Functional Modules Overview

- Resume & JD Upload Interface
- Resume Parsing (PDF/DOCX to Text)
- NLP-based Keyword Extraction
- Similarity Score Generation (TF-IDF / BERT)
- Filter, Sort, and View Results
- Export Shortlist as PDF/CSV

3. Tools and Technologies

3.1 Programming Languages

Python, HTML, CSS, JavaScript

3.2 Frameworks / Libraries

Flask, spaCy, BERT, sklearn, Pandas, pdfminer/similar

3.3 Databases / Storage

PostgreSQL, CSV (temporary storage), JSON

3.4 IDEs / Platforms

VS Code, Jupyter Notebook

3.5 APIs / External Integrations

HuggingFace Transformers, Resume Parsing APIs (optional)

3.6 Hosting / Deployment Tools

GitHub, Localhost, optional Heroku/Netlify for demo

4. Project Plan

4.1 Module-wise Feature Breakdown

Module	Description
Resume & JD Upload	Allow users to upload resumes and job descriptions
Parsing Engine	Extracts text from PDF/DOCX formats
NLP Processing	Extracts relevant skills, keywords, and entities
Score Generator	Calculates similarity using vector methods
Result Interface	Displays matched results in ranked format
Export Module	Allows export of matched results in CSV or PDF

4.2 Week-wise Development Timeline

Module	Planned Weeks	Expected Deliverables
Module 1: Resume & JD Upload	Week 3–4	Upload page with backend handling
Module 2: Resume Parser Engine	Week 4–5	Working text extraction pipeline
Module 3: NLP-Based Processing	Week 5–6	Keyword extraction + JD parsing
Module 4: Similarity Score Logic	Week 6–7	Match score logic implementation
Module 5: Result Display + Filtering	Week 7–8	Web UI for result presentation
Module 6: Export & Final Integration	Week 9–10	Export + integrated dashboard
Final Documentation & Report	Week 11	Final formatted report
Final Demo + Evaluation	Week 12	Presentation & evaluation submission

4.3 Expected Outcomes

- Functional resume shortlisting tool
- NLP-powered match score engine
- User-friendly interface for HR and job seekers
- Exportable shortlists and detailed reports

4.4 Risk Factors / Assumptions

- Parsing errors from PDF formats
- Accuracy of similarity score algorithms
- Resource limitation for large-scale dataset testing

5. Literature Survey

S. No.	Source Title / Tool	Key Idea	Project Relevance
1	"Automated Resume Screening using Natural Language Processing" – IEEE (2020)	Uses NLP and cosine similarity to match resumes with job descriptions.	Establishes a base methodology for resume shortlisting using similarity scoring.
2	"Job2Vec: Learning Job Descriptions Representations from Job Posting Data" – ACL (2021)	Proposes semantic vector representation for job postings.	Supports the use of vector embeddings like BERT in matching resumes with JDs.
3	HuggingFace Transformers – BERT	Open-source library for pretrained language models.	Enables context-aware semantic similarity between resume and JD.
4	spaCy NLP Library Documentation	Industrial-grade NLP processing for tokenization, POS tagging, and named entity recognition.	Provides core resume parsing and keyword extraction capabilities.
5	"AI-Based Recruitment System: Challenges and	Highlights pros and cons of using AI in hiring.	Validates the real-world importance and

	Opportunities" – ACM Digital Library (2019)		ethical aspects of AI-based screening.
6	Scikit-learn Documentation – TF-IDF Vectorizer	Traditional keyword-based vectorizer.	Supports comparison between semantic and statistical similarity techniques.
7	"Parsing Unstructured Resumes using Deep Learning Models" – Springer (2022)	Deep learning approach to extract structured data from CVs.	Useful for designing the resume parsing module with DL if required.
8	ResumeParser (Open-source tool)	Tool to extract structured data from resumes in various formats (PDF, DOCX).	Reference for implementing or improving your resume upload and parsing module.
9	LinkedIn Job Match Recommendation Model	Proprietary matching algorithm using profile/job data.	Offers conceptual guidance for scoring and filtering resumes.
10	Google's BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (2018)	Introduced BERT model for capturing deep context in text.	Underpins the semantic match model in the SkillScan system.

References (APA Style)

1. Ahmad, M., & Khan, A. (2020). Automated resume screening using natural language processing. *2020 International Conference on Computational Intelligence and Knowledge Economy (ICCIKE)*, IEEE. <https://doi.org/10.1109/ICCIKE48598.2020.9312232>
2. Zhang, W., Xu, H., & Zhao, L. (2021). Job2Vec: Learning job description representations from job posting data. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*. <https://aclanthology.org/2021.acl-long.395>
3. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
4. Explosion AI. (2023). *spaCy 3.0: Industrial-strength natural language processing in Python* [Documentation]. <https://spacy.io>

5. Sharma, R., & Bhatia, M. P. S. (2019). AI-based recruitment system: Challenges and opportunities. *Communications of the ACM*, 62(11), 54–61. <https://doi.org/10.1145/3359550>
6. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <http://jmlr.org/papers/v12/pedregosa11a.html>
7. Gupta, R., & Jain, S. (2022). Parsing unstructured resumes using deep learning models. *Journal of Intelligent & Fuzzy Systems*, 42(2), 1817–1825. <https://doi.org/10.3233/JIFS-212420>
8. Omkarpathak27. (2021). *ResumeParser: Extract information from resumes in PDF or DOCX formats* [GitHub Repository]. <https://github.com/OmkarPathak/ResumeParser>
9. Zhang, Y., Zhao, B., Chen, Y., & Wang, Y. (2019). AI for job matching at LinkedIn. *LinkedIn Engineering Blog*. <https://engineering.linkedin.com/blog/2019/ai-job-matching-linkedin>
10. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*. <https://arxiv.org/abs/1810.04805>